

A Few-Shot Class-Incremental Learning Method for Network Intrusion Detection

Lei Du¹, Zhaoquan Gu, *Member, IEEE*, Ye Wang, Le Wang², *Member, IEEE*, and Yan Jia

Abstract—With the rapid development of information technologies, the security of cyberspace has become increasingly serious. Network intrusion detection is a practical scheme to protect network systems from cyber attacks. However, as new vulnerabilities and unknown attack types are constantly emerging, only a few samples of such attacks can be captured for analysis, which cannot be handled by the existing detection methods deployed in real systems. To handle this problem, we propose a few-shot class-incremental learning method called Branch Fusion Strategy based Network Intrusion Detection (BFS-NID for short), which can continuously learn new attack classes with only a few samples. BFS-NID includes a feature extractor module and a branch classifier learning module. The feature extractor module uses a vision transformer to learn better feature representations in a self-supervised manner, and the parameters of the feature extractor are fixed to avoid catastrophic forgetting when the model learns incrementally. The branch classifier learning module sets re-projection for different branch sessions to enhance the feature representation ability between classes and employs a branch fusion strategy to associate the context of learned attack classes with new classes in different sessions. We conducted extensive experiments on two popular network intrusion detection benchmark datasets (CIC-IDS2017 and CSE-CIC-IDS2018) and the results demonstrate that BFS-NID surpasses the baselines and achieves the best performance.

Index Terms—Cyber security, network intrusion detection, few-shot class-incremental learning.

I. INTRODUCTION

WITH the emergence of innovative information technologies, a large number of devices and applications have been adopted in various areas, such as autonomous driving systems [16], Internet of Things smart devices [25],

and industrial cyber-physical systems [23]. These devices and applications bring convenience to the modern daily life of humans, but they also have become the target of attackers, which significantly threaten the assets and data security of individuals, enterprises, and even the country [1], [6]. Hence, detecting and blocking malicious attacks in time is critical for ensuring the security of networked systems.

Network intrusion detection is a practical scheme for detecting attacks. Most of the traditional network intrusion detection methods utilize rule-based methods to detect intrusions, and the detection rule is generated from known attack samples [2], [24]. For example, Syrius is proposed in [2] which uses benign and malicious traffic samples to generate Suricata's detection rules. Bayesian networks are adopted in [24] to learn attack patterns and translate them into IF-THEN detection rules. These methods can detect known network attacks accurately and efficiently, but cannot deal with variants of known attacks and new types of attacks. In practice, attackers often use a variety of techniques to cover up their attack behaviors in order to evade detection, thereby making detection more difficult. For example, obfuscation techniques have been used to generate variants from known attacks that evade traditional detection methods [4]. In addition, attackers might utilize the latest vulnerability to launch attacks, such as the *0-day* and *n-day* vulnerabilities. These unknown (new) attacks cannot be effectively detected since they have not appeared before [29]. Even though some cyber security teams will analyze such new attacks once they appear, the number of samples is quite small.

To handle these problems, many researchers attempt to employ Artificial Intelligence (AI) technologies for network intrusion detection [7], [12], [37], [38], [40], [41], as AI technologies have reached or even surpassed human-like performance in many tasks. Regarding the problem of attack variants, there has been some significant progress [47], but it is quite limited to the problem of detecting new attacks.

The existing methods of detecting new attacks can be divided into two categories: anomaly-based binary classification [7], [38] and fine-grained multi-classification [12], [41]. Anomaly detection methods use benign samples for modeling, and any behavior patterns that deviate from expectations are identified as anomalies. For example, Regularization and autoencoders are introduced in [7] to improve potential representations for anomaly detection. OnlineBPCA is proposed in [38] which converts network traffic into a two-dimensional tensor and analyzes the changes of principal directions for anomaly detection. However, anomaly detection is rarely

Manuscript received 27 April 2023; revised 7 September 2023 and 4 November 2023; accepted 9 November 2023. Date of publication 13 November 2023; date of current version 15 April 2024. This work was supported in part by the Major Key Project of PCL(PCL2022A03), National Natural Science Foundation of China (Grant No. 62372137), and Guangdong Provincial Key Laboratory of Novel Security Intelligence Technologies (2022B121010005). The associate editor coordinating the review of this article and approving it for publication was V. Fodor. (*Corresponding author: Zhaoquan Gu.*)

Lei Du, Zhaoquan Gu, and Yan Jia are with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen 518055, Guangdong, China, and also with the Department of New Networks, Peng Cheng Laboratory, Shenzhen 518055, Guangdong, China (e-mail: 21b951040@stu.hit.edu.cn; guzhaoquan@hit.edu.cn; jia yan2020@hit.edu.cn).

Ye Wang is with the School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen 518055, Guangdong, China, and also with the College of Computer, National University of Defense Technology, Changsha 410073, China (e-mail: wangye2020@hit.edu.cn).

Le Wang is with the Cyberspace Institute of Advanced Technology, Guangzhou University, Guangzhou 510006, China (e-mail: wangle@gzhu.edu.cn).

Digital Object Identifier 10.1109/TNSM.2023.3332284

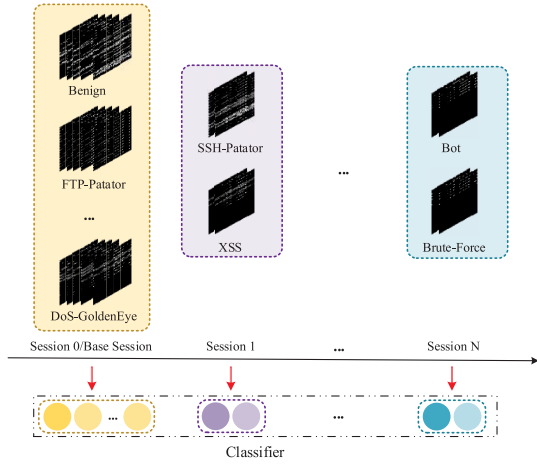


Fig. 1. The setting of FSCIL for network intrusion detection. FSCIL can continuously learn new attack classes from sessions without forgetting the former attack classes and without overlapping attack classes in sessions. The base session as the initial model training phase contains sufficient samples, while the subsequent sessions as the class increment phase learn new attack classes with only a few samples. After learning of the session, the classifier model is able to classify among all seen classes.

deployed in realistic situations due to the inability to provide detailed diagnostic information about attack types and the high false positive rate [40]. Fine-grained detection methods are trained and evaluated using supervised learning based on pre-defined classes. In the study [12], a system called FT-GCN was proposed that leverages the underlying traffic flow pattern and converts intrusion detection into a special node classification task for detecting intrusions. PBCNN is a few-shot learning method [41] that uses a byte-packet-flow data structure to automatically extract features to detect attack classes with only a few samples. However, these methods require the attack class of the detected samples to appear in the pre-defined classes [37]. In the realistic network, new attack classes are constantly appearing and these detection models cannot cope with the classification of the new attack samples.

In recent years, Few-Shot Class-Incremental Learning (FSCIL) has emerged in several research fields [14], [30], [44], [45], such as in the computer vision area [26], [46], which for incrementally learning new classes from a few of labeled samples without forgetting previously learned classes [19]. However, few of them study network intrusion detection. We envision that combating the new attacks requires establishing a network intrusion detection system with FSCIL to incrementally learn new attack classes with only a few samples without forgetting previously learned classes. When handling network intrusion detection using FSCIL, it is important to maintain the distinction between the emerging new classes and the learned classes. The task set is shown in Fig. 1. The FSCIL suffers from a serious class imbalance problem between the base session and the new session, which not only makes the model prone to over-fitting to new classes but also exacerbates the catastrophic forgetting of learned attack class knowledge.

Previous works have made extensive efforts to resist catastrophic forgetting from different perspectives. For example, optimizing the feature representation space with the learning

ability of the model [30], pre-assigning virtual prototypes in the feature representation space [45], or using a freeze pre-trained unified feature extractor [44]. These FSCIL methods employ a standard supervised manner to pre-training models. The parameters of the feature extractor are frozen to obtain a better feature representation to resist catastrophic forgetting. However, the feature representation of the feature extractor relies on large-scale labeled samples. Furthermore, as incremental learning proceeds, the classifier accommodates an increasing number of classes. We find that the feature extractor extracts highly similar feature representations for different class samples.

In this paper, we propose a few-shot class-increment learning method for network intrusion detection with a branching fusion strategy, called BFS-NID. The proposed method is able to learn a generalization feature representation and continually keep with the discriminative ability between all seen attack classes. In detail, the method consists of two modules: a feature extractor module and a branch classifier learning module. The feature extractor module uses a vision transformer to learn the behavioral patterns of network traffic samples and is pre-trained in a self-supervised manner without relying on sample labels. The branch classifier learning module includes two stages to learn branch classifiers for different sessions. For each session, the branch classifier has a re-projection to enhance the discriminability of the original feature representation of different classes. The base session with ample samples is used for the training of the initial branch projection layer. The remaining session projection layers are fine-tuned according to the initial projection layer parameters. The learned branch classifiers and new attack samples jointly optimize a new branch projection layer. Further, the branch fusion strategy associates the contextual relationship between sessions and assists the training and inference of the model.

The contribution of this work is summarized as follows:

- To the best of our knowledge, we are the first to propose a few-shot class-incremental learning method for network intrusion detection using grayscale images, which is effective against catastrophic forgetting and overfitting.
- We explore the ability of self-supervised manner feature representation for unseen classes in the cybersecurity area and propose combining branch fusion strategy to enforce the distinction between new classes and seen classes in different sessions.
- We propose a concise and intuitive representation of network traffic that converts raw network traffic into grayscale images while maintaining the time-series relationship between packets of bi-flows.

We also conduct extensive experiments on two benchmarks, the CIC-IDS2017 and CSE-CIC-IDS2018 datasets, and the results demonstrate that BFS-NID significantly outperforms the baselines with significant advantages.

The rest of this paper is organized as follows. Section II discusses the related work about few-shot class-incremental learning and network intrusion detection. Section III introduces the preliminaries and Section IV describes the representation of network traffic. Section V introduces the proposed method BFS-NID. Afterward, the experimental

results and discussion are shown in Section VI. Finally, Section VII concludes this paper and prospects for future work.

II. RELATED WORK

This section presents related work on network intrusion detection, class incremental learning, and few-shot class-incremental learning.

A. Network Intrusion Detection

Network Intrusion detection methods can be divided into rule-based detection methods, anomaly-based detection methods, and fine-grained-based detection methods. The rule-based detection method uses the known attack patterns in the rule sets to generate alarms. The capability of detection is limited by the rule sets. Therefore, it is necessary to continuously generate and update rules to improve the quality of the rule sets. Due to manually generating detection rules for attack samples is time-consuming and error-prone. Some researchers propose automated methods to generate detection rules, such as deep packet inspection techniques [2], and Bayesian networks [24]. However, these methods are unable to detect unknown attacks. Anomaly-based detection methods use benign samples to build models and any biases are identified anomalies. An anomaly detection method is proposed in [7] to improve latent representations through regularization and autoencoders. OnlineBPFA is proposed in [38] which can fast and accurate detection anomalies. IDERES [28] adapts network profiling and machine learning algorithms to identify malicious attack traffic in IoT networks.

To effectively detect unknown anomalies, a probabilistic method [29] is proposed through constructing samples as a Bayesian network for identifying zero-day attacks. PPPL [17] uses the domain adaptation method to build a network intrusion detection system for detecting unseen anomalies. However, anomaly-based detection methods cannot provide detailed diagnostic information about attacks and have a high false positive rate. Other methods use few-shot learning to identify new attacks [8]. A meta-learning framework based method FC-Net is proposed for new attack detection by comparing pairs of samples to distinguish classes [37]. DPNSA [9] proposes a dynamic prototype network based on sample adaptation for detecting malware attacks with few samples. However, existing methods can only recognize new attack patterns by comparison and cannot incrementally learn new attack types that are constantly emerging. To alleviate this problem, IDFSIL [35] proposes a few-shot class-incremental learning strategy that uses meta-learning for initially training on old data and trains new classifiers by fine-tuning new attack class data. Then, the logit layers of all classifiers are jointly optimized on a small amount of old and new class data using a fully connected network. This strategy requires retaining and replaying small amounts of old data, and the amount of retained data will increase as the number of sessions increases. In contrast to IDFSIL, the method proposed in this work uses only new data for incremental learning sessions on the base model and employs re-projection to improve the discrimination

of different classes of samples and avoid the overhead of storing old data.

B. Class-Incremental Learning

The goal of class-incremental learning (CIL) is to continuously learn new classes without forgetting the learned classes. The main challenge is catastrophic forgetting. To overcome this challenge, several regularization methods have been proposed. Lwf [21] uses regularization terms to preserve class knowledge that has been learned. MAS [3] and EWC [18] are proposed to use regularization terms to constrain the range of variation of important parameters when learning new sessions. SDC improves the performance of the model by incorporating semantic drift compensation [42]. Other methods dynamically adjust the structure of the model to adapt to the learning of new sessions [39]. From another perspective, existing methods can be divided into exemplar-based methods and nonexemplar-based methods. Exemplar-based methods store some of the learned class examples and replay them when learning new sessions. The nearest neighbor classifier iCaRL [27] is combined with a distillation loss to prevent forgetting when learning new sessions. Considering the imbalance between the exemplar classes of the base session and new sessions, LUCIR [15] imposes constraints on the exemplars of the base session classes when learning new classes. A dynamic reinforced memory management method RMM [22] optimizes the incremental phase and different classes. FOSTER [36] proposes a two-stage learning paradigm for adaptive learning of new classes. However, the CIL method leads to poor performance due to the limited number of training samples in new sessions [46].

C. Few-Shot Class-Incremental Learning

Few-shot class-incremental learning was recently proposed, which leverages the knowledge learned from large datasets to help learn new classes with few samples [19], [43]. TOPIC proposes [33] to preserve the topology structure between learned classes and new classes during incremental learning. To tackle the overfitting that may be caused by the few samples of new classes, FSLL [26] proposes to select a few parameters for updating. FACT [45] proposed a forward-compatibility method to compress the embedding of learned classes by virtual prototypes in preparation for learning new classes. LIMIT [46] constructs a generalization feature space for the new session and utilizes a calibration module to calibrate the semantic relationship between the old classifier and the new prototype. C-FSCIL [14] uses hyperdimensional embedding and loss functions to preserve the orthogonal relationship between class vectors. Other methods [30], [44] find that using pre-trained backbone networks can achieve good performance even for new classes that have not been seen before. On the basis of this discovery. A coupled training strategy CEC [44] is proposed, which uses a graph model to adapt the classifier to an incremental process across different sessions. F2M [30] proposes to find a flat minimal region in the base session and finetune the new session within the region. Different

from existing methods, our proposed method performs a re-projection on the original feature embedding of different sessions to improve the discriminability of classes and uses a branching strategy to fuse classifiers from different branch sessions.

III. PRELIMINARIES

FSCIL for network intrusion detection aims to achieve incremental learning of new attack classes with only a few available samples, without forgetting the learned knowledge of classes. Let $D = \{D^0, D^1, \dots, D^i, \dots, D^N\}$ denotes a sequence train datasets of different sessions, where $D^i = \{(x_n, y_n)\}_{n=1}^{|D^i|}$ is the train dataset of session i . x_n is a sample of class $y_n \in C^i$. C^i is the label space of session i . There are no overlapped classes across different sessions, i.e., $\forall i \neq j, C^i \cap C^j = \emptyset$. It is noted that each session contains new classes of attacks identified based on factors such as time, user/host context, network connections, or operational scenarios. The first session provides a large train dataset with sufficient samples and classes, which is also denoted as the base session. The subsequent sessions only have a few new attack samples and classes, e.g., $\exists i > 0$, session D^i contains C new attack classes in the training dataset, and per class has K attack samples. Hence it is often called a C -way K -shot. For each session i training, only the dataset of session D^i is available, the train datasets of former sessions are not available. At the evaluation, the model evaluates the test dataset from all encountered classes, i.e., $C = \cup_{s=0}^s C^s$. FSCIL faces continuous learning of new attack classes with only a few samples. The main challenge is catastrophic forgetting, where the learned classes are forgotten as the model learns new attack classes. Especially for new types of attacks with few attack samples, the problem of catastrophic forgetting is more serious due to the phenomenon of overfitting.

Problem Definition: To ensure the security of cyberspace, we aim to develop an ideal detection model using the continuously generated flow data D in real-world network applications, which can incrementally learn new attack classes C^i with few attack samples, and maintain the detection performance of the model.

IV. NETWORK TRAFFIC REPRESENTATION

Network traffic is a complex data structure, according to the OSI model, the network model has seven layers for information transmission and exchange. The minimum transmission unit of network traffic is a packet, which includes one or more headers and a payload. For example, an HTTP protocol packet is composed of a MAC layer header, an IP layer header, a TCP layer header, an HTTP layer header, and a payload. The network traffic transmitted by the network is captured and further stored as Pcap files for analysis to protect the target networked system from malicious attacks. The analysis granularity of network traffic is very important in cybersecurity research. Summarized in [10], the five commonly used granularities are flows, bidirectional flows (bi-flows), TCP connections, services, and hosts. This work uses the bi-flows as analysis granularity. The bi-flows can be

extracted from network traffic by using quintuple data (source IP, destination IP, source port, destination port, protocol), and the source IP - port and destination IP - port can be interchangeable. According to the quintuple, the captured network traffic or Pcap files can split into a large number of bi-flows, and each one contains many packets. It needs to be further divided into attack or benign samples for analysis and modeling. We adopt the same process as [41], including timeout and periodic reset strategies. The timeout strategy refers to the information exchange being terminated when the communication link has been waiting for a long time. The threshold is set to 64 seconds. Considering only the timeout strategy may still lead to a sample containing too many packets. A periodic reset strategy is further used and the threshold is set to 120 seconds. In order to facilitate the processing of the model, the sample needs to be converted into a unified representation format. There are two mainstream ways of utilizing the divided samples: statistical features of the sample or raw bytes data in the packets of the sample. The former way requires different feature extraction rules for different protocols that are designed by cybersecurity experts, which lack adaptability and feature engineering is complex and time-consuming. Furthermore, this way pays more attention to the information in the packet header and ignores the payload [37]. However, malicious information in a successful attack may be in the protocol header or a payload. e.g., SQL injection attacks use benign protocol headers and a payload containing malicious SQL statements. The latter represents network traffic as visual images, such as bi-flows of raw network traffic converted to several color images [5] or to an RGB image [37]. These works demonstrate the promise of the converted images for network intrusion detection, where different classes of images still have different visual representations [47]. However, the existing network traffic representation uses color images with complex structures or does not take into account the time-series relationship between bi-flows packets, which seriously hinders further research in image-based network intrusion detection.

This section proposes a concise and intuitive representation of network traffic, which converts raw network traffic into grayscale images while maintaining the time-series relationship between packets of bi-flows. Specifically, each sample consists of a different number of packets, and each packet consists of variable-length bytes. The representation of the sample uses the first M packets, each packet is represented by the first B bytes. In addition, it is also necessary to check whether the sample meets the requirements of the number of packets $\geq M$ and the length of packet bytes $\geq B$, respectively. If it is insufficient, fill in with 0; otherwise, the excess part will be truncated. Then, the B bytes in the packet are reshaped into a matrix $\in R^{h' \times w}$, the M packets are arranged according to the arrival time series of the packet, and the final sample is converted into a grayscale image $\in R^{1 \times (h' \times M) \times w}$. For the convenience of processing, the grayscale image is set as a square $h = w = h' \times M$. M and B are set to 16 and 256 respectively, which is consistent with [37], [41]. The conversion process is shown in Fig. 2. Taking a DoS-GoldenEye attack sample as an example, each

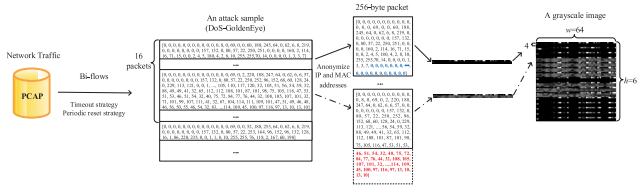


Fig. 2. The process of converting network traffic to grayscale images. First, the captured network traffic is segmented into bi-flows. Then, a timeout strategy and a periodic reset strategy are used to split the bi-flows to obtain samples. After that, packets or bytes are padded or truncated (bytes are padded in denoted blue and truncated in denoted red) and IP and MAC addresses are anonymized. Finally, the grayscale image is converted by a reshape of B bytes and an organization of time-series relationships of M packets.

packet is converted into a matrix $\in R^{4 \times 64}$, and the attack sample is represented by a grayscale image $\in R^{1 \times 64 \times 64}$. The code is publicly available¹.

Note that there are three key points in the process. Firstly, the first M packets - the first B bytes can represent the whole sample. These packets containing the most important information are usually transmitted first, and the appropriate bytes are sufficient to represent most of the information in the packet. Second, the sample data conforms to the grayscale image format. The byte of the packet is encoded hexadecimal with 8 bits, the numerical type of each byte corresponds to a pixel value in a grayscale image, ranging from 0-255. Third, the converted grayscale image does not contain sensitive information. The MAC and IP addresses have special significance in cyberspace and will leak user privacy. For security and privacy reasons, anonymization is used to replace the MAC and IP addresses in packets with all 0s.

V. PROPOSED METHOD: BFS-NID

In this section, we introduce a few-shot class-incremental learning method for network intrusion detection combined with branch fusion strategies. The overview of the method consists of two modules: *the feature extractor module* and *the branch classifier learning module*, as depicted in Fig. 3. We first present a pre-trained feature extractor using a self-supervised manner. Then, we describe how the branch classifier helps the learning of the base session and the new sessions. To coordinate the training and inference of branch classifiers, we further propose a fusion strategy.

A. The Feature Extractor Module

Recently research demonstrated that pre-trained feature extractors in a standard supervised manner only with base session data can achieve competitive performance in the FSCIL task [44]. However, this manner relies on a large number of labeled samples to train a robust feature extractor for better feature representation. Self-supervised learning utilizes the intrinsic structural information of samples for representation learning without labeled samples. Therefore, we employ a self-supervised manner to pre-train the feature extractor. Generally, there are two common network architectures used to extract feature embedding from image samples: Convolutional Neural Networks (CNN) and Vision Transformers (ViT) [11]. In

Section IV, the converted samples are not directly equivalent to grayscale images due to the time-series nature of network traffic. Thus, the feature extractor should be able to learn two aspects of information including the feature of each packet and the contextual time series relationship between packets in bi-flows samples. CNN learns the pixel-level feature representation of the grayscale image, which cannot capture the time-series relationship of bi-flows packets. The ViT divides the grayscale image into multiple patches, and the sequence relationship between patches is represented by position encoding. It can meet network traffic data requirements: the packets correspond to the divided patches and the time-series relationship of the packets corresponds to the position encoding between the patches. Therefore, we choose ViT as the backbone network of the feature extractor. Furthermore, for better generalization, we employ a patch mask data augmentation technique to train the feature extractor [13]. As depicted in Fig. 3 (a), the task of a self-supervised feature extractor is to reconstruct grayscale image samples. The grayscale image samples are divided into patches, and then patches and position embedding are fed into the model. Each patch size is $\in R^{4 \times 64}$, which corresponds to one packet of bi-flows packets.

B. The Branch Classifier Learning Module

The pre-trained feature extractor has different optimization goals for applications, where the self-supervised pre-trained backbone network is a grayscale image reconstruction task, and the downstream task is an intrusion detection task. Therefore, it is very necessary to optimize the parameters of the feature extractor before being applied to intrusion detection. We follow the way of the Prototype Network [32] as the classifier, which is slightly different from the conventional classifier when calculating the classification probability. The Prototype Network adopts the idea of metric. In each episode, the data fed to the model are the support set and query set. The support set contains samples from multiple classes and is used to compute the prototype vectors of classes. The class prototype is the average of the feature embedding of the class samples, calculated as:

$$p_i^k(D_k^i, \phi) = \frac{1}{|D_k^i|} \sum_{(x_n, y_n) \in D_k^i} f_\phi(x_n) \quad (1)$$

p_i^k represents the prototype vector of the k -th class in the session i , $p_i^k \in R^{1 \times d}$, D_k^i is the support set, and f_ϕ is the feature extractor. The predicted label of a query set sample is the class with the highest similarity score calculated with class prototype vectors. In the FSCIL task, only the training data is available, instead of dividing it into support and query sets, we use it as a query set to train the feature extractor, which is then used as a support set to update the class prototype vectors. Furthermore, we implement more simple training strategies rather than episodic meta-learning. For this reason, we utilize randomly initialized prototype vectors and feature embedding of the sample to train the feature extractor and compute the similarity score.

Consider the procedure of FSCIL using a pre-trained feature extractor. The base session contains a large number of classes

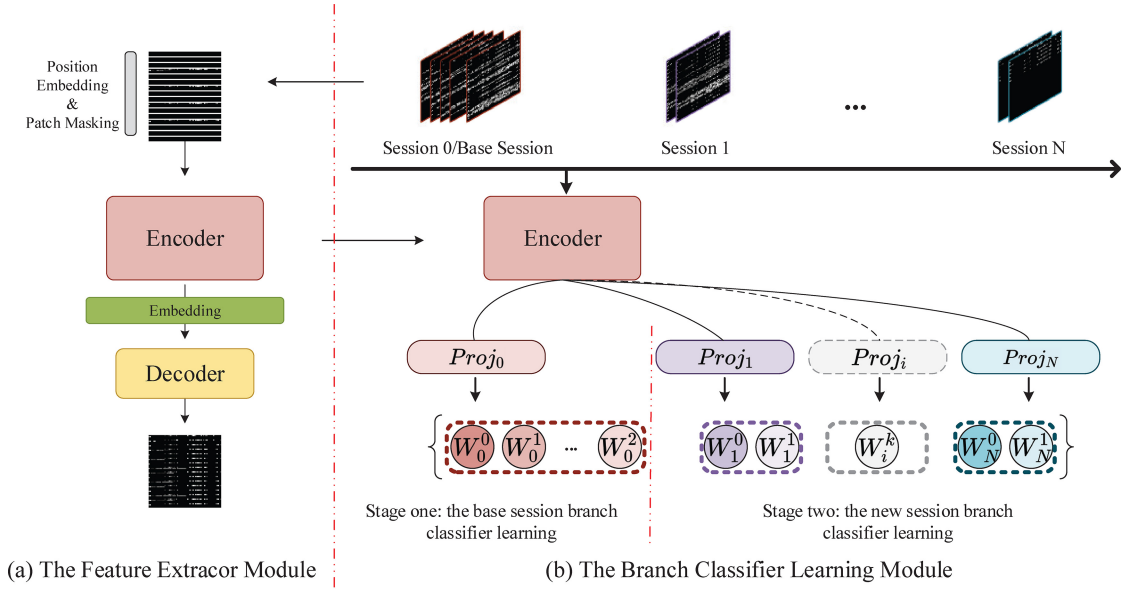


Fig. 3. Illustration of our method. BFS-NID mainly includes two modules: (a) The feature extractor module to learn pre-trained backbone network with basic session data in a self-supervised manner, and (b) the branch classifier learning module, which contains two stages base and new session branch classifier learning, to learn the parameters of the projection layer for different sessions and uses a fusion strategy to help model training and inference.

and data, while the new session has few classes and limited samples. The problem of class imbalance between the base session and the new session will exacerbate catastrophic forgetting. To handle this issue, the base session data is used to pre-train and fine-tune the feature extractor, which is frozen during the learning process of subsequent new sessions. To this end, the learning of all sessions is divided into two stages: the base session stage and the new session stage. In the base session stage, the pre-trained feature extractor is used to extract feature embedding of samples, as follows:

$$fea_x = f_\theta(x) \quad (2)$$

f_θ is the pre-trained feature extractor, and the feature embedding of the sample x is denoted as fea_x . Then, the cosine distance *cosine* is used as a metric function to calculate the similarity score between the feature embedding of the sample and the prototype vectors of classes in the base session:

$$score(x, P_i) = \bigcup_{j=0}^{|P_i|} cosine(fea_x, p_i^j) \quad (3)$$

P_i is the class prototype vectors in the session i , $|P_i| = |C^i|$. p_i^j represents the prototype vector of the j -th class in the session i . The training loss for the base session is calculated using the Cross-Entropy loss function, and the model is optimized using the backpropagation algorithm:

$$J(D^i, P_i, f_\theta) = - \sum_{n=1}^{|D^i|} y_n \log(score(fea_{x_n}, P_i)) \quad (4)$$

y_n is the ground truth of the sample x_n . When the base session training is completed, class prototype vectors are updated via Eq. (1). Further, these class prototype vectors are stored in the fully connected layer W for the classification of the i -th session classifier. Such as $i = 0$, $W_0^j = p_0^j$, where W_0 belongs to $R^{n \times d}$, n is the number of classes in the base session, and d is the dimension of the class prototype vector. In the subsequent

sessions, the sample feature embedding is used to compute prototype vectors of the classes in the new session, which are continuously merged into the previous classifiers. After the new session is learned, the fused classifier can be used to classify all seen classes.

In Eq. (2), a frozen feature extractor is used to extract feature embedding of samples for different classes across different sessions. However, if the feature extractor is designed to extract feature embedding for seen class samples, it is inevitable to impair the discriminative representation of different classes in new sessions. To relieve this problem, a re-projection for different sessions is proposed to improve the discriminative of feature embedding for different classes within the session. As shown in Fig. 3 (b), these sessions are divided into two stages, the base session branch classifier stage and the new session branch classifier stage, each session with a projection layer $Proj$. In the learning stage of the base session branch classifier, finetuning the feature extractor and training the projection layer $Proj_0$. During the new session branch classifier learning stage, only training the projection layer $Proj_i$ of the new session:

$$fea'_x = \begin{cases} Proj_0(f_\theta(x)) & x \in D^i, i = 0 \\ Proj_i(f_\theta(x)) & x \in D^i, i \neq 0 \end{cases} \quad (5)$$

The projection layer $Proj$ is a linear transformation that transforms the original feature embedding of samples into another feature representation space. Since only a few samples of classes are available in the new session, the $Proj_0$ of the base session is used as an initialization for the $Proj_i$ of the new session and then fine-tuned to accommodate the learning of classes in the new session.

We need to further modify the formula to enable incremental training across different sessions. In Eq. (3) and (4), the measure of the similarity score and the training of the model are session-independent, new sessions did not participate in training. In addition, in the new session learning, only use

the samples in the current session, and cannot utilize the class prototype vector that has been learned in the previous session. However, an incremental classifier should break down the boundaries between sessions. It should continuously adapt to new classes, and maintain the ability to discriminate from learned classes across different sessions. Hence, we metric the similarity score between the feature embedding of samples and the class prototype vectors in previous sessions during the re-projection training of the new session. P represents all classes that have been seen already.

$$J'(D^i, P, f_\theta, Proj_i) = - \sum_{n=1}^{|D^i|} y_n \log(\text{score}(\text{fea}'_{x_n}, P)) \quad (6)$$

During the incremental learning and inference process of the model, the re-projection of the branch classifier needs to be continuously adjusted to select feature embedding of the sample corresponding to the session. To achieve this goal, we propose a branch fusion strategy that fuses the similarity scores of different branch classifiers. Specifically, the similarity scores between the feature embedding of different session re-projections of the query sample and known class prototypes are first calculated. Then the similarity scores with the corresponding class prototypes in different sessions are selected. Further, the repository *Repo* is used to store the parameters of projection layers in order to facilitate management. The process of the algorithm is summarized as Alg. 1. During the branch classifier training process in the base session, the class prototype vectors and the projection layer are randomly initialized, and then these parameters are fine-tuned and updated by the similarity score. In new sessions, the class prototype vectors need to be randomly initialized and updated. The projection layer is initialized with the base session projection layer and updated by computing a similarity score with learned class prototypes. During branch classifier inference, the feature embedding of query samples is fed into the corresponding projection layers, and similarity scores are computed using it and seen class prototype vectors. The similarity scores of different sessions are fused into a similarity score matrix, and the label predicted by the classifier is the one with the highest score.

VI. EXPERIMENT

In this section, we evaluate the proposed BFS-NID on two popular datasets in network intrusion detection. First, we present dataset statistics and experimental details. Then, we comprehensively evaluate the effectiveness of the BFS-NID. Finally, we compare with baseline methods, including finetune, CIL, and FSCIL methods.

A. Datasets and Experiment Setting

Datasets. We used CIC-IDS2017 and CSE-CIC-IDS2018 as benchmark datasets [31], which are widely used in network intrusion detection research. The CIC-IDS2017 dataset collects five days of network traffic, and the CSE-CIC-IDS2018 contains ten days of network traffic, both of which include benign traffic and 14 classes of malicious traffic. Each dataset provides

Algorithm 1 Our Proposed Method

Require: The sequence sessions datasets D , pre-trained model f_θ , a randomly initialized prototype vector matrix W .

Ensure: Network Intrusion Detection Classifier.

- 1: // Training the base branch classifier.
- 2: Randomly initialize the parameters of the projection layer $Proj_0$;
- 3: **for** epoch $p = 1, 2, \dots$ **do**
- 4: **for** batch $b = 1, 2, \dots$ **do**
- 5: Sample training samples from the base session D^0 ;
- 6: Extract feature embedding of samples by Eq. (5);
- 7: Calculate the similarity score using Eq. (3);
- 8: **end for**
- 9: Update $f_\theta, Proj_0$ with the loss J' defined in Eq. (6);
- 10: **end for**
- 11: Update each prototype vector of the classes in the session;
- 12: Save the prototype vectors of the base session to W_0 and $Proj_0$ to *Repo*.
- 13: // Incremental training of the new branch classifier.
- 14: **for** session $i = 1, 2, \dots, N$ **do**
- 15: Initialize the parameters of the projection layer $Proj_i$ with $Proj_0$ from the repository *Repo*;
- 16: **for** epoch $p = 1, 2, \dots$ **do**
- 17: Sample training samples from the new session D^i ;
- 18: Extract feature embedding of samples by Eq. (5);
- 19: Calculate the similarity score Eq. (3);
- 20: Update $Proj_i$ with the loss J' defined in Eq. (6);
- 21: **end for**
- 22: Save the prototype vectors of the new session to W_i and $Proj_i$ to *Repo*.
- 23: **end for**

Pcaps files of raw network traffic and CSV files of statistical features extracted by CICFlowMeter. As mentioned before, we use the Pcaps files of the raw traffic for experiments. Limited by the number of specific attack samples and label mismatches, we choose classes with sufficient samples for experiments. As shown in Table I, the settings of datasets are as follows:

- There are 12 classes in CIC-IDS2017, which are divided into 8 classes in the base session and 4 classes in the new sessions. The 4 classes are divided into two sessions, each session contains 2 classes.
- The CSE-CIC-IDS2018 includes 14 classes, the base session contains 8 classes and 6 classes are further divided into 2 new sessions, each session contains 3 classes.
- In the training phase, each class of the base session contains 2000 samples, while each class of the new session only has 5 samples. In the testing phase, each class from different sessions has 100 samples for evaluation.

Implementation Details: We use the Vitlarge model [11] as the backbone network for the experiments of CIC-IDS2017 and CSE-CIC-IDS2018. We modify the parameters of the backbone network, the dimension of the feature embedding is changed to 512, and the patch is modified to 4×64 . We follow the MAE [13] to pre-train the backbone network

TABLE I
THE SETTINGS OF DIFFERENT SESSIONS

CIC-IDS2017			CSE-CIC-IDS2018		
Base session	Session 1	Session 2	Base session	Session 1	Session 2
Benign, DDoS, PortScan, DoS-Slowloris, DoS-SlowHTTPTest, DoS-Hulk, DoS-GoldenEye, FTP-Patator	SSH-Patator, Bot	Web-BruteForce, Web-XSS	Benign, DDoS-HOIC, DDoS-LOIC-HTTP, Bot, FTP-BruteForce, DoS-Hulk, DoS-SlowHTTPTest, DoS-Slowloris	BruteForce-Web, BruteForce-XSS, DDoS-LOIC-UDP	DoS-GoldenEye, Infiltration, Bruteforce-SSH

TABLE II
ABLATION ANALYSIS ON TWO BENCHMARK DATASETS TO ANALYZE THE EFFECTIVENESS OF DIFFERENT MODULES IN BFS-NID. FE IS THE FEATURE EXTRACTOR PRE-TRAINING MODULE, REPROJ IS THE RE-PROJECTION, AND BFS IS THE BRANCH FUSION STRATEGY

FE	REPROJ	BFS	CIC-IDS2017				CSE-CIC-IDS2018			
			Acc. in each session (%) $\uparrow$			PD $\downarrow$	Acc. in each session (%) $\uparrow$			PD $\downarrow$
			0	1	2		0	1	2	
✓	✗	✗	99.88	97.00	88.67	11.21	99.88	88.91	87.00	12.88
✓	✓	✗	100.0	97.90	92.92	7.08	100.0	91.36	90.00	10.0
✓	✓	✓	100.0	98.10	93.17	6.83	100.0	91.73	90.14	9.86

using a self-supervised manner, modify the decoder feature embedding dimension to 384, and keep the patch mask ratio at 75%. Our network is built using the deep learning library PyTorch and optimized by SGD. In the base session branch classifier training stage, the CIC-IDS2017 dataset is iterated 40 epochs with a learning rate of 0.05, and the CSE-CIC-IDS2018 dataset is iterated 80 epochs with a learning rate of 0.002. The training stage of the new session branch classifier is iterated 100 epochs on both datasets with a learning rate of 0.0002, and the batch size is 128.

Evaluation Protocol: To quantitatively evaluate the performance and resist forgetting the ability of different methods, follow [46], we use Top 1 accuracy and performance dropping rate (PD) as evaluation indicators. Top 1 accuracy is used to indicate the model prediction accuracy, and the accuracy of the first session is represented by A. PD is used to measure the accuracy drop rate of the sessions, that is, $PD = A - B$, where A is the accuracy of the base session, and B is the accuracy of the last incremental session. The model with a smaller PD means a better anti-forgetting effect. In addition, we also use multi-class evaluation indicators to measure the false positives and false negatives of network intrusion detection, which will be reported through the confusion matrix.

B. Experimental Analysis

We conducted multiple experiments to evaluate the effectiveness of modules in the proposed method. Further, we compare and analyze the pre-trained models of the standard supervised manner and the self-supervised manner. Finally, the false positives and false negatives of the proposed method are analyzed during the incremental learning process.

Ablation Analysis: We first performed an ablation analysis on two benchmark datasets to observe the effectiveness of different modules, as shown in Table II. We gradually

introduce our designs to observe their impact on predictive performance. For example, only use the self-supervised features extractor (FE), use the re-projection (REPROJ), and further combine the branch fusion strategy (BFS). From Table II, we can observe that the self-supervised model has competitive incremental learning ability. But it still faces serious overfitting and catastrophic forgetting problems. In the last session, the PD performance dropped by 11.21% and 12.88% on the CIC-IDS2017 and CSE-CIC-IDS2018 datasets respectively. After adding the REPROJ, the accuracy of different sessions is improved, and the PD performance is improved by 4.13% and 2.88% on the two datasets, respectively. It should be noted that although the backbone network uses a vision transformer as an encoder network, these parameters will be frozen after the base session branch is trained. The main reason for the model performance improvement is the establishment of corresponding re-projection feature embedding for different sessions. In addition, when the BFS is considered, the prediction performance of the model will be further improved. We can infer that considering the prototype vector relationship when optimizing the re-projection feature embedding helps to improve the distinction between the old and new classes. These results imply that our proposed method can achieve stable performance improvements on the few-shot class-incremental learning task.

To further observe the predictive performance improvement brought by the REPROJ, we visualize the feature embedding of test samples and the learned decision boundary in a low-dimensional space using t-SNE [34]. As depicted in Fig. 4, the classes encountered during the incremental learning sessions of two network intrusion detection datasets CIC-IDS2017 and CSE-CIC-IDS2018 are visualized. Fig. 4 (a) and (c) show the decision boundary using the self-supervised manner of the feature extractor. The decision boundary with the REPROJ is visualized in Fig. 4 (b) and (d). By comparing before and after adding REPROJ, we can observe that the sample

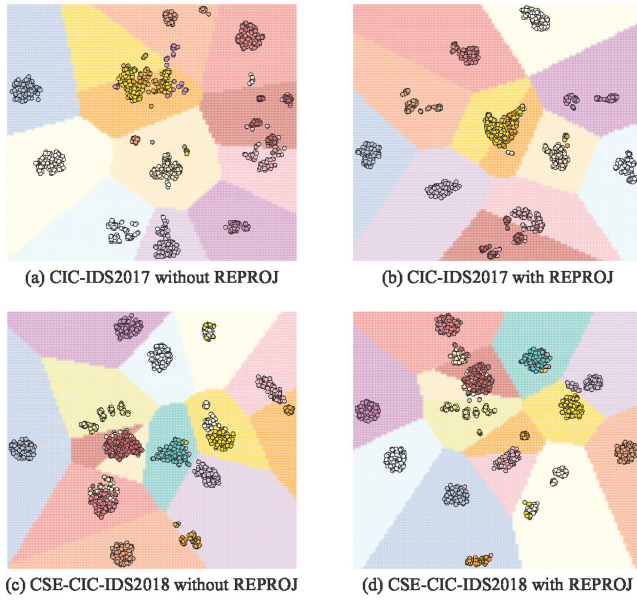


Fig. 4. t-SNE [34] visualization of data embedding and classifier decision boundaries before and after the REPROJ on two benchmarks. The shaded area depicts the decision boundary for each class.

feature embedding of different classes in the feature space is clearly separated, while the sample feature embedding of the same class is tightly embedded or clustered. This indicates the REPROJ is able to update class prototype vectors and calibrate decision boundaries through re-projection feature representation in the new session branch classifier learning stage, even though a few samples of new incremental classes. For example, a 2-way 5-shot branch incremental task contains 2 classes, and only 5 samples of each class are available.

We demonstrate the predictive performance of without and with branch fusion strategy in the last session using a confusion matrix with normalization. As shown in Fig. 5, we present the confusion matrix on two datasets, with bold blue lines separating the classes of the base session and the new classes of subsequent sessions. The x-axis of the confusion matrix indicates the ground truth, and the y-axis indicates the prediction by the model. The diagonal elements of the confusion matrix represent classification accuracy. Fig. 5 (a) and (c) are the normalization confusion matrix of the datasets CIC-IDS2017 and CSE-CIC-IDS2018 without adding the branch fusion strategy. The normalization confusion matrix of the branch fusion strategy is shown in Fig. 5 (b) and (d). By comparison, we can observe that the overall false positive rate of all classes has decreased. For example, in the CSE-CIC-2018, the Benign class was misclassified to the other six classes when the branch fusion strategy was not added. The number of misclassified to other classes was four after the addition. The figure demonstrates the branch fusion strategy can effectively maintain and utilize the discriminative relationship between the learned class prototype and the new class knowledge, thereby helping the re-projection to complete the adaptive update of the new class feature embedding.

Analysis of model pre-training manner: Further, we compare the performance impact of BFS-NID employing different

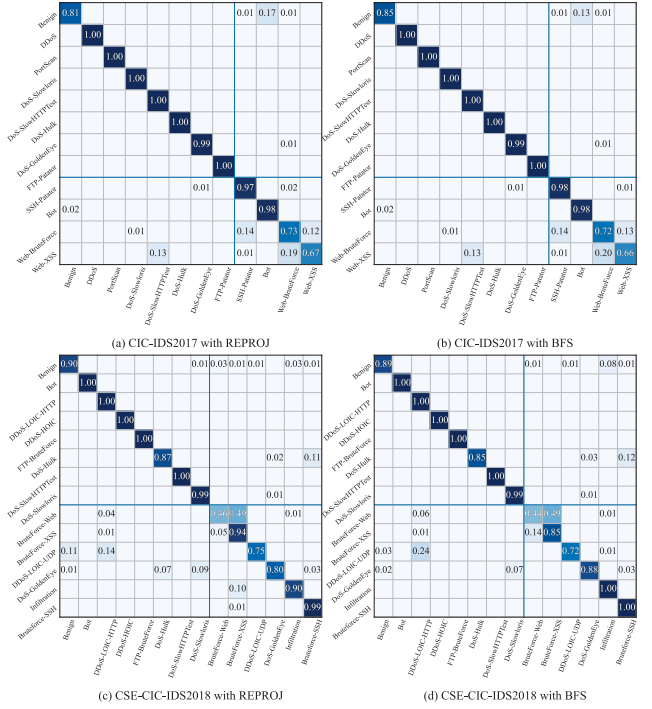


Fig. 5. Normalized confusion matrices with REPROJ and BFS on two benchmark datasets. We use bold blue lines to separate regions of the base session classes and incremental classes of new sessions.

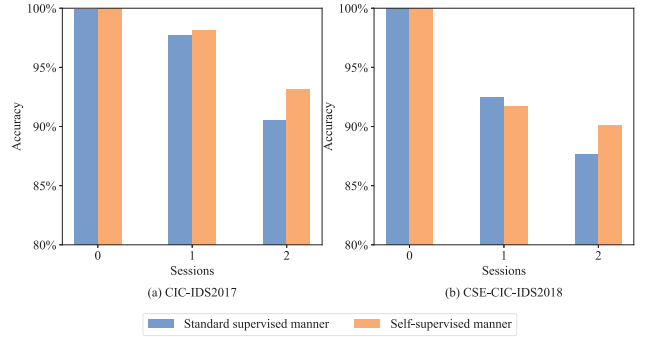


Fig. 6. Comparison of different training manners for pre-trained model on two benchmarks: (a) CIC-IDS2017 and (b) CSE-CIC-IDS2018.

pre-training manners. As shown in Fig. 6, we evaluate the performance of the same backbone network on different sessions using two pre-training manners, a standard supervised manner, and a self-supervised manner. From the figure, we can observe that the self-supervised manner model has a better resist-forgetting ability. In the CIC-IDS2017, the self-supervised manner of the model maintains superior performance and PD in different sessions. In the CSE-CIC-IDS2018, the model of the self-supervised manner has an explicit advantage in the PD, but the model of the standard manner in session 1 has better predictive performance. We infer there are two main reasons. First, the self-supervised pre-training model can obtain feature embedding with better generalization through representation learning. In addition, the data augmentation with patch mask utilizes the transformer structure to simulate the context sequence relationship of different packets to enhance the generalization of the model

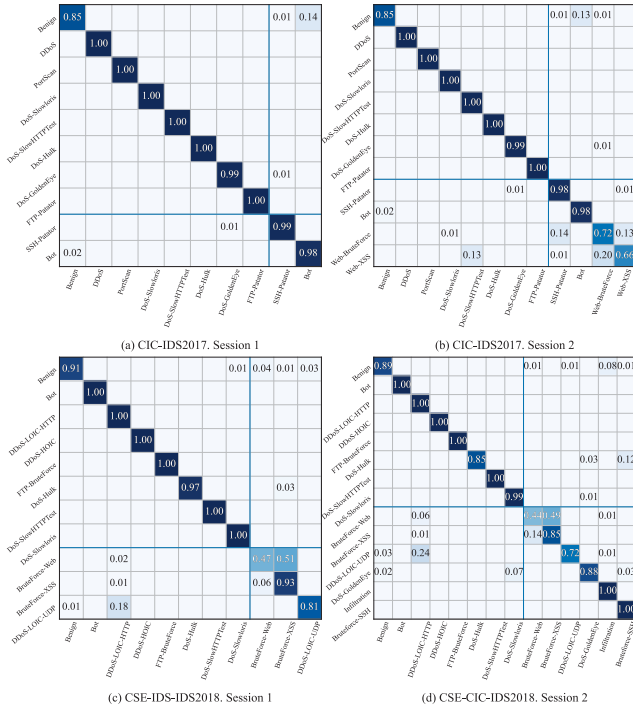


Fig. 7. Normalized confusion matrices on two benchmarks between two sessions. We use bold blue lines to separate regions of the base session classes and incremental classes of new sessions.

further. Second, the standard supervised pre-training model relies on a large amount of training data and requires the training data to maintain diversity to obtain better model generalization. On the contrary, when the training data lacks diversity, higher classification accuracy can be achieved, but the generalization ability of incremental learning is lost. From the perspective of the incremental learning task, compared with the standard pre-training manner the self-supervised pre-training manner can effectively resist the catastrophic forgetting problem.

False Positive and False Negative Analysis: In the application of network security, false positives and negatives are important indicators that affect whether the intrusion detection system can be deployed. A large number of false negatives will lead to the inability to detect intrusion in time, and important assets or data will be stolen. Many false positives will lead to excessive manpower and time costs for analysis. To this end, we present normalized confusion matrices to evaluate false positives and negatives in different incremental learning tasks. Since 100% accuracy was achieved in the base session phase, we only plot the confusion matrix for the subsequent two incremental phases. As shown in Fig. 7, we report the confusion matrices on two network intrusion detection datasets, with bold blue lines separating the classes of the base session and the new classes of subsequent sessions. The x-axis of the confusion matrix indicates the ground truth, and the y-axis indicates the prediction by the model. The diagonal elements of the confusion matrix represent the classification accuracy of the model.

First, we analyze the false negatives of BFS-NID. In CIC-IDS2017, the false negative rate of session 1 and session

2 remains the same at 2%. In CSE-CIC-IDS2018, the false negative rate of session 1 was 1%, and session 2 increased to 5%. It can be observed that BFS-NID has a maximum false negative rate of 5%. Then, we analyze the false positives in the case of benign and attack binary classification. In CIC-IDS2017, benign samples in session 1 are misclassified into two attack classes SSH-Patator and Bot with 1% and 14% probabilities, respectively. The number of misclassified attack classes increased to three in session 2, namely SSH-Patator 1%, Bot 13%, and WEB-BruteForce 1%. In CSE-CIC-IDS2018, benign samples in session 1 were misclassified into four attack classes: DOS-Slowloris 1%, BruteForce-Web 4%, BruteForce-XSS 1%, DDoS-LOIC-UDP 3%. In session 2, the number of misclassified attack classes remained the same, but the attack classes were modified to BruteForce-Web 1%, DDoS-LOIC-UDP 1%, Infiltration 8%, and BruteForce-SSH 1%. According to observations, BFS-NID produces a large false positive rate, 15% of benign samples that are misclassified as attacks. We infer the main reason is benign data contain a wider diversity, which is very easy to confuse with the behavior patterns of existing attack class samples.

Finally, we analyze the false positives among attack classes. Alerts triggered by attacks provide detailed descriptive information for subsequent analysis by cybersecurity experts. Therefore, we not only need to pay attention to whether an attack has occurred but also need fine-grained attack class description information. It can be seen from the figure that most of the attack classes maintain good distinguishability, but some attack classes are seriously confused. In Fig. 7 (b), the attack samples of Web-BruteForce are misclassified as SSH-Patator and Web-XSS with 14% and 13%, respectively. The Web-XSS samples are misclassified as DoS-SlowHTTPTest at 13% and Web-BruteForce at 20%. In Fig. 7 (d), 49% of the attack samples of BruteForce-Wed are misclassified as BruteForce-XSS, and 14% of BruteForce-XSS are misclassified as BruteForce-Wed. In addition, DDoS-LOIC-UDP was misclassified as DDoS-LOIC-HTTP with 24%. We observe that these severely confused attack classes share some of the same attack tricks and techniques, making it difficult to distinguish them from the feature representation level. For example, in CIC-IDS2017, both brute force and XSS belong to Web attacks but use different attack methods. In CSE-CIC-IDS2018, both DDoS-LOIC-HTTP and DDoS-LOIC-UDP are distributed denial-of-service attacks, but they use different network protocols.

C. Comparison With Baseline Methods

We compare the performance of BFS-NID with five existing baseline methods on two benchmark datasets. The results are in Fig. 8 and details are shown in Table III. From Fig. 8, we observe that our proposed method achieves the highest accuracy across different sessions and maintains the lowest PD during the overall incremental learning. It is particularly noteworthy that the accuracy rate of benchmark datasets exceeds the existing baseline methods by 1%, and the PD exceeds 1%. The finetune suffers from severe catastrophic forgetting problems when learning new class knowledge because it

TABLE III
COMPARISON WITH BASELINE METHODS ON CIC-IDS2017 AND CSE-CIC-IDS2018 DATASETS

Method	CIC-IDS2017				CSE-CIC-IDS2018			
	Acc. in each session (%) $\uparrow$			PD $\downarrow$	Acc. in each session (%) $\uparrow$			PD $\downarrow$
	0	1	2		0	1	2	
Finetune	99.75	78.80	47.08	52.67	100.0	82.00	63.14	36.86
iCaRL [27]	99.75	96.00	77.58	22.17	100.0	87.82	74.86	25.14
FOSTER [36]	99.75	96.80	83.00	16.75	100.0	84.45	80.21	19.79
IDFSCIL [35]	99.88	96.70	89.17	10.71	100.0	90.73	83.93	16.07
LIMIT [46]	99.63	97.80	90.58	9.05	100.0	91.18	85.71	14.29
C-FSCIL [14]	99.88	97.80	92.00	7.88	100.0	89.64	89.14	10.86
BFS-NID (Ours)	100.0	98.10	93.17	6.83	100.0	91.73	90.14	9.86

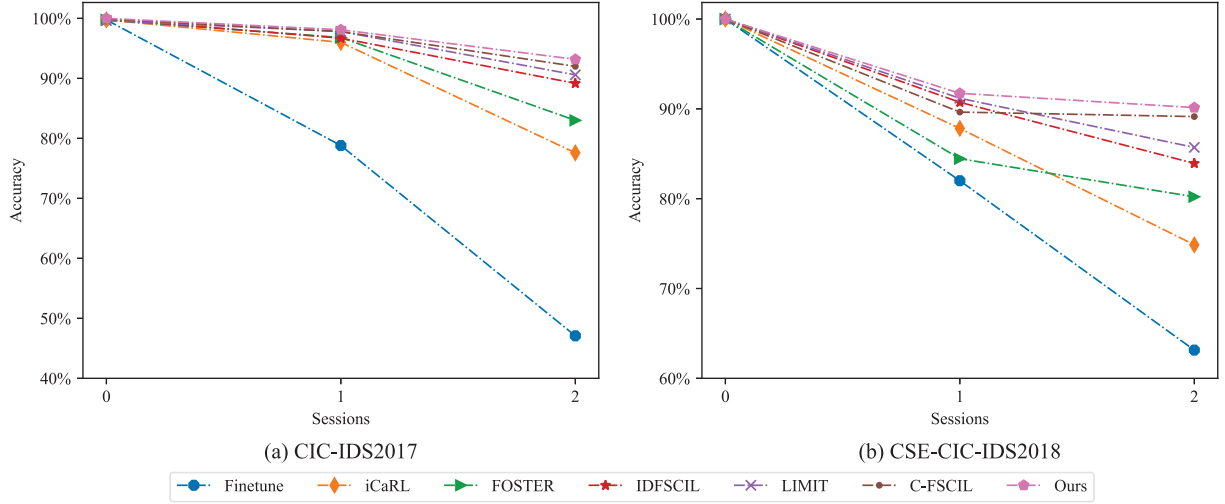


Fig. 8. Comparison with baselines on two benchmarks: (a) CIC-IDS2017 and (b) CSE-CIC-IDS2018.

TABLE IV
COMPARISON OF DIFFERENT METHODS IN PRECISION, RECALL, AND F1-SCORE

Method	CIC-IDS2017			CSE-CIC-IDS2018		
	Precision	Recall	F1-score	Precision	Recall	F1-score
Finetune	0.4041	0.4708	0.4053	0.5406	0.6314	0.5553
iCaRL [27]	0.7993	0.7758	0.7798	0.7349	0.7486	0.7036
FOSTER [36]	0.8391	0.8300	0.8192	0.8208	0.8021	0.7716
IDFSCIL [35]	0.8884	0.8917	0.8886	0.8530	0.8393	0.8358
LIMIT [46]	0.9061	0.9058	0.9040	0.8773	0.8571	0.8305
C-FSCIL [14]	0.9249	0.9200	0.9190	0.9103	0.8914	0.8785
BFS-NID (Ours)	0.9313	0.9317	0.9295	0.9099	0.9014	0.8979

does not consider the learned class knowledge. The class incremental learning methods iCaRL [27] and FOSTER [36], can maintain the balanced relationship between the learned class and the new class knowledge when learning new class knowledge, but require many available samples during incremental learning. We observe that these methods face serious overfitting problems when only a few samples are available. IDFSCIL [35], LIMIT [46] and C-FSCIL [14] are few-shot class incremental learning methods that use standard pre-training models to achieve competitive performance. We also compared the precision, recall, and f1-score of the last session with the baseline methods, as shown in Table IV. Compared with them, we think the BFS-NID advantage mainly lies in

TABLE V
COMPARISON OF DIFFERENT MODELS. ALL EXPERIMENTS RUN ON NVIDIA TESLA P100. FLOPs ARE TESTED WITH ONE SAMPLE AND THE INFERENCE SPEED (SAMPLES/SECOND) IS THE AVERAGE OF 100 RUNS WITH A BATCH SIZE OF 100 AFTER HARDWARE WARM-UP

Model	FLOPs	Memory usage	Inference speed
Finetune	142.44M	3128.91M	2229
iCaRL [27]	142.44M	3113.43M	2122
FOSTER [36]	284.87M	3734.39M	1456
IDFSCIL [35]	1287.23M	3082.63M	784
LIMIT [46]	142.43M	3075.12M	2282
C-FSCIL [14]	2096.31M	3000.39M	1653
BFS-NID (Ours)	1287.49M	2998.84M	481

two aspects. The self-supervised manner pre-trained model provides better feature representation ability. In addition, BFS-NID performs a re-project feature embedding for different incremental stages, which relieves the feature embedding of samples of other classes from being embedded into an approximate feature space. Overall, our proposed method outperforms the baseline methods and achieves the best model prediction performance on two network intrusion detection benchmark datasets.

We also observed some flaws in the computational performance of BFS-NID. In Table V, we show the floating-point operations (FLOPs), memory usage, and inference speed for all models in the last session. The FLOPs is often used to

measure the theoretical complexity of an algorithm or model. We observe that BFS-NID has high complexity and poor inference performance. Although C-FSCIL has higher FLOPs, it still demonstrates superior inference speed than BFS-NID. The reason is that FLOPs is an indirect measure that ignores important factors such as memory access cost and model execution parallel. Furthermore, IDFSCIL and BFS-NID use the same backbone network architecture, show similar FLOPs, and have the bottom two inference speeds. We can infer that the reason for the low computational performance of BFS-NID is that the backbone network architecture used and the multiple branches implemented are not conducive to parallel operations. An alternative solution may involve alternative backbone network architectures such as lightweight ViT [20].

VII. CONCLUSION

Network intrusion detection is one of the most critical ways to find and detect intrusion. However, existing network intrusion detection methods should not be able to incremental learning of new attack classes with a few samples. The main challenge is catastrophic forgetting and overfitting. In this paper, we propose BFS-NID for the few-shot class-incremental learning of network intrusion detection. The BFS-NID contains the pre-train feature extractor module and the branch classifier learning module. The pre-train feature extractor using a self-supervised is used to learn powerful feature representation ability. In the branch classifier learning module, re-projection is used to enhance the feature embedding, and a branch fusion strategy helps to correlate different sessions and build up the contextual relationship between class prototypes. We conduct experiments on two network intrusion detection datasets and the results demonstrate that BFS-NID achieves the best predictive performance. In future work, we will explore fine-grained network intrusion detection in the open world, and verify the detection performance of the adopted network traffic representation method on the encrypted network traffic.

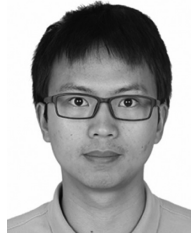
REFERENCES

- [1] E. Anthi, L. Williams, M. Słowińska, G. Theodorakopoulos, and P. Burnap, "A supervised intrusion detection system for smart home IoT devices," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 9042–9053, Oct. 2019, doi: [10.1109/IIOT.2019.2926365](#).
- [2] L. Alcantara, G. Padilha, R. Abreu, and M. D'Amorim, "Syrius: Synthesis of rules for intrusion detectors," *IEEE Trans. Rel.*, vol. 71, no. 1, pp. 370–381, Mar. 2022, doi: [10.1109/TR.2021.3061297](#).
- [3] R. Aljundi, F. Babiloni, M. Elhoseiny, M. Rohrbach, and T. Tuytelaars, "Memory aware synapses: Learning what (not) to forget," in *Proc. 15th Eur. Conf. Comput. Vision (ECCV)*, vol. 11207, 2018, pp. 144–161, doi: [10.1007/978-3-030-01219-9_9](#).
- [4] J. M. Beaver, C. T. Symons, and R. E. Gillen, "A learning system for discriminating variants of malicious network traffic," in *Proc. 8th Annu. Cyber Secur. Inf. Intell. Res. Workshop*, 2013, pp. 1–4, doi: [10.1145/2459976.2460003](#).
- [5] G. Bendiab, S. Shiales, A. Alruban, and N. Kolokotronis, "IoT Malware network traffic classification using visual representation and deep learning," in *Proc. 6th IEEE Conf. Netw. Softw. (NetSoft)*, Ghent, Belgium, 2020, pp. 444–449, doi: [10.1109/NetSoft48620.2020.9165381](#).
- [6] M. M. Alani and A. I. Awad, "An intelligent two-layer intrusion detection system for the Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 683–692, Jan. 2023, doi: [10.1109/TII.2022.3192035](#).
- [7] V. L. Cao, M. Nicolau, and J. McDermott, "Learning neural representations for network anomaly detection," *IEEE Trans. Cybern.*, vol. 49, no. 8, pp. 3074–3087, Aug. 2019, doi: [10.1109/TCYB.2018.2838668](#).
- [8] Y. Chai, J. Qiu, L. Yin, L. Zhang, B. B. Gupta and Z. Tian, "From data and model levels: Improve the performance of few-shot malware classification," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4248–4261, Dec. 2022, doi: [10.1109/TNSM.2022.3200866](#).
- [9] Y. Chai, L. Du, J. Qiu, L. Yin, and Z. Tian, "Dynamic prototype network based on sample adaptation for few-shot malware detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 5, pp. 4754–4766, May 2023, doi: [10.1109/TKDE.2022.3142820](#).
- [10] A. Dainotti, A. Pescapé, and K. C. Claffy, "Issues and future directions in traffic classification," *IEEE Netw.*, vol. 26, no. 1, pp. 35–40, Jan./Feb. 2012, doi: [10.1109/MNET.2012.6135854](#).
- [11] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. 9th Int. Conf. Learn. Represent. (ICLR)*, 2021, pp. 1–21.
- [12] X. Deng, J. Zhu, X. Pei, L. Zhang, Z. Ling, and K. Xue, "Flow topology-based graph convolutional network for intrusion detection in label-limited IoT networks," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 1, pp. 684–696, Mar. 2023, doi: [10.1109/TNSM.2022.3213807](#).
- [13] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conf. Comput. Vision Pattern Recognit. (CVPR)*, New Orleans, LA, USA, 2022, pp. 15979–15988, doi: [10.1109/CVPR52688.2022.01553](#).
- [14] M. Hersche, G. Karunaratne, G. Cherubini, L. Benini, A. Sebastian, and A. Rahimi, "Constrained few-shot class-incremental learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, New Orleans, LA, USA, 2022, pp. 9047–9057, doi: [10.1109/CVPR52688.2022.00885](#).
- [15] S. Hou, X. Pan, C. C. Loy, Z. Wang, and D. Lin, "Learning a unified classifier incrementally via rebalancing," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Long Beach, CA, USA, 2019, pp. 831–839, doi: [10.1109/CVPR.2019.00092](#).
- [16] X. Hou, J. Zhang, C. He, Y. Ji, J. Zhang, and J. Han, "Autonomous driving at the handling limit using residual reinforcement learning," *Adv. Eng. Informat.*, vol. 54, Oct. 2022, Art. no. 101754, doi: [10.1016/j.aei.2022.101754](#).
- [17] M. J. Hashemi, E. Keller, and S. Tizpaz-Niari, "Detecting unseen anomalies in network systems by leveraging neural networks," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 2515–2528, Sep. 2023, doi: [10.1109/TNSM.2022.3220775](#).
- [18] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," 2016, arXiv: 1612.00796.
- [19] A. Kukleva, H. Kuehne, and B. Schiele, "Generalized and incremental few-shot learning by explicit learning and calibration without forgetting," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Montreal, QC, Canada, 2021, pp. 9000–9009, doi: [10.1109/ICCV48922.2021.00889](#).
- [20] Y. Li et al., "Rethinking vision transformers for MobileNet size and speed," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 16889–16900.
- [21] Z. Li and D. Hoiem, "Learning without forgetting," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 12, pp. 2935–2947, Dec. 2018, doi: [10.1109/TPAMI.2017.2773081](#).
- [22] Y. Liu, B. Schiele, and Q. Sun, "RMM: Reinforced memory management for class-incremental learning," *Proc. 35th Annu. Conf. Neural Inf. Process. Syst.*, NeurIPS 2021, pp. 3478–3490.
- [23] Y. Lian, Q. Yang, Y. Liu, and W. Xie, "A spatio-temporal constrained hierarchical scheduling strategy for multiple warehouse mobile robots under industrial cyber-physical system," *Adv. Eng. Informat.*, vol. 52, Apr. 2022, Art. no. 101572, doi: [10.1016/j.aei.2022.101572](#).
- [24] Q. Liu, H. B. Keller, and V. Hagenmeyer, "A Bayesian rule learning based intrusion detection system for the MQTT communication protocol," in *Proc. 16th Int. Conf. Availab. Rel. Secur.*, Vienna, Austria, 2021, pp. 1–10, doi: [10.1145/3465481.3470046](#).
- [25] K. Liu et al., "Security analysis of mobile device-to-device network applications," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2922–2932, Apr. 2019, doi: [10.1109/IIOT.2018.2877174](#).
- [26] P. Mazumder, P. Singh, and P. Rai, "Few-shot lifelong learning," in *Proc. 35th AAAI Conf. Artif. Intell.*, 2021, pp. 2337–2345.
- [27] S. A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "iCaRL: Incremental classifier and representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Honolulu, HI, USA, 2017, pp. 5533–5542, doi: [10.1109/CVPR.2017.587](#).
- [28] J. R. Rose et al., "IDERS: Intrusion detection and response system using machine learning and attack graphs," *J. Syst. Architect.*, vol. 131, Oct. 2022, Art. no. 102722.
- [29] X. Sun, J. Dai, P. Liu, A. Singhal, and J. Yen, "Using Bayesian networks for probabilistic identification of zero-day attack paths," *IEEE Trans. Inf. Foren. Secur.*, vol. 13, no. 10, pp. 2506–2521, Oct. 2018, doi: [10.1109/TIFS.2018.2821095](#).

- [30] G. Shi, J. Chen, W. Zhang, L.-M. Zhan, and X.-M. Wu “Overcoming catastrophic forgetting in incremental few-shot learning by finding flat minima,” in *Proc. 35th Annu. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2021, pp. 6747–6761.
- [31] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. 4th Int. Conf. Syst. Secur. Privacy, (ICISSP)*, Funchal, Portugal, 2018, pp. 108–116.
- [32] J. Snell, K. Swersky, and R. Zemel, “Prototypical networks for few-shot learning,” in *Proc. 31st Annu. Conf. Neural Inf. Process. Syst.*, 2017, pp. 4077–4087.
- [33] X. Tao, X. Hong, X. Chang, S. Dong, X. Wei, and Y. Gong, “Few-shot class-incremental learning,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, 2020, pp. 12180–12189, doi: [10.1109/CVPR42600.2020.01220](https://doi.org/10.1109/CVPR42600.2020.01220).
- [34] L. Van Der Maaten and G. Hinton, “Visualizing data using t-SNE,” in *Proc. J. Mach. Learn. Res.*, vol. 9, 2008, pp. 2579–2605.
- [35] T. Wang, Q. Lv, B. Hu, and D. Sun, “A few-shot class-incremental learning approach for intrusion detection,” in *Proc. Int. Conf. Comput. Commun. Netw. (ICCCN)*, Athens, Greece, 2021, pp. 1–8, doi: [10.1109/ICCCN52240.2021.9522260](https://doi.org/10.1109/ICCCN52240.2021.9522260).
- [36] F. Y. Wang, D. W. Zhou, H. J. Ye, and D.-C. Zha, “FOSTER: Feature boosting and compression for class-incremental learning,” in *Proc. 17th Eur. Conf. Comput. Vision (ECCV)*, vol. 13685, 2022, pp. 398–414, doi: [10.1007/978-3-031-19806-9_23](https://doi.org/10.1007/978-3-031-19806-9_23).
- [37] C. Xu, J. Shen, and X. Du, “A method of few-shot network intrusion detection based on meta-learning framework,” *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3540–3552, 2020, doi: [10.1109/TIFS.2020.2991876](https://doi.org/10.1109/TIFS.2020.2991876).
- [38] K. Xie et al., “On-line anomaly detection with high accuracy,” *IEEE/ACM Trans. Netw.*, vol. 26, no. 3, pp. 1222–1235, Jun. 2018, doi: [10.1109/TNET.2018.2819507](https://doi.org/10.1109/TNET.2018.2819507).
- [39] S. Yan, J. Xie, and X. He, “DER: Dynamically expandable representation for class incremental learning,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Nashville, TN, USA, 2021, pp. 3013–3022, doi: [10.1109/CVPR46437.2021.00303](https://doi.org/10.1109/CVPR46437.2021.00303).
- [40] J. Yang, X. Chen, S. Chen, X. Jiang, and X. Tan, “Conditional variational auto-encoder and extreme value theory aided two-stage learning approach for intelligent fine-grained known/unknown intrusion detection,” *IEEE Trans. Inf. Forensics Secur.*, vol. 16, 2021, pp. 3538–3553, doi: [10.1109/TIFS.2021.3083422](https://doi.org/10.1109/TIFS.2021.3083422).
- [41] L. Yu, et al., “PBCNN: Packet bytes-based convolutional neural network for network intrusion detection,” *Comput. Netw.*, vol. 194, Jul. 2021, Art. no. 108117, doi: [10.1016/j.comnet.2021.108117](https://doi.org/10.1016/j.comnet.2021.108117).
- [42] L. Yu et al., “Semantic drift compensation for class-incremental learning,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Seattle, WA, USA, 2020, pp. 6980–6989, doi: [10.1109/CVPR42600.2020.00701](https://doi.org/10.1109/CVPR42600.2020.00701).
- [43] H. Zhao, Y. Fu, M. Kang, Q. Tian, F. Wu, and X. Li, “MgSvF: Multi-grained slow vs. fast framework for few-shot class-incremental learning,” *IEEE Trans. Pattern Anal. Mach. Intell.*, early access, Dec. 9, 2021, doi: [10.1109/TPAMI.2021.3133897](https://doi.org/10.1109/TPAMI.2021.3133897).
- [44] C. Zhang, N. Song, G. Lin, Y. Zheng, P. Pan, and Y. Xu, “Few-shot incremental learning with continually evolved classifiers,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Nashville, TN, USA, 2021, pp. 12450–12459, doi: [10.1109/CVPR46437.2021.01227](https://doi.org/10.1109/CVPR46437.2021.01227).
- [45] D.-W. Zhou, F.-Y. Wang, H.-J. Ye, L. Ma, S. Pu, and D.-C. Zhan, “Forward compatible few-shot class-incremental learning,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, New Orleans, LA, USA, 2022, pp. 9036–9046, doi: [10.1109/CVPR52688.2022.00884](https://doi.org/10.1109/CVPR52688.2022.00884).
- [46] D.-W. Zhou, H.-J. Ye, L. Ma, D. Xie, S. Pu, and D.-C. Zhan, “Few-shot class-incremental learning by sampling multi-phase tasks,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 11, pp. 12816–12831, Nov. 2023, doi: [10.1109/TPAMI.2022.3200865](https://doi.org/10.1109/TPAMI.2022.3200865).
- [47] J. Zhao, R. Masood, and S. Seneviratne, “A review of computer vision methods in network security,” *IEEE Commun. Surveys Tuts.*, vol. 23, no. 3, pp. 1838–1878, 3rd Quart., 2021, doi: [10.1109/COMST.2021.3086475](https://doi.org/10.1109/COMST.2021.3086475).



Lei Du received the master's degree from the Hebei University of Science and Technology in 2021. He is currently with the School of Computer Science and Technology, Harbin Institute of Technology (Shenzhen), China. He is also with Peng Cheng Laboratory, Shenzhen, China. His research focuses on cyberspace security, machine intelligence, and open-world learning.



include cyberspace security, cyber range, big data analysis, and artificial intelligence security.

Zhaoquan Gu (Member, IEEE) received the bachelor's and Ph.D. degrees in computer science from Tsinghua University in 2011 and 2015, respectively. He was a Professor and the Associate Dean of the Cyberspace Institute of Advanced Technology, Guangzhou University, China. He is currently a Professor with the School of Computer Science and Technology, Harbin Institute of Technology (Shenzhen), China. He is also a Professor with the Department of New Networks, Peng Cheng Laboratory, Shenzhen, China. His research interests



Ye Wang received the Ph.D. degree in information and mathematical science from Victoria University, Australia, in 2019. She is currently a Research Fellow with the Harbin Institute of Technology (Shenzhen) and an Assistant Professor with the National University of Defense Technology. Her research interests include social network analysis, information content security, and knowledge graph reasoning.



a member of the China Computer Federation.

Le Wang (Member, IEEE) received the Ph.D. degree in computer science from NUDT, China, in 2008. He is currently an Associate Professor with the Cyberspace Institute of Advanced Technology, Guangzhou University as well as a Part Time Associate Professor with Pengcheng Laboratory, China. He has authored 20+ journal and conference papers in these areas, as well as a book in Chinese. His current research interests include network and big data security. He also served as the co-chair and the general chair of international conferences. He is



Chairman at the Chinese Information Processing Society of China and the Chair of the Specialty Committee of Big Search in Cyberspace.

Yan Jia is a Professor with the Harbin Institute of Technology (Shenzhen). She is also a Professor with the Department of New Networks, Peng Cheng Laboratory, Shenzhen, China. As a Principal Investigator, she has undertaken more than 20 national projects, including the National Key Project of 863 Program and the National Natural Science Foundation of China. Her research interests cover big data analysis, artificial intelligence, online social network analysis, and security situation awareness and analysis in cyberspace. She serves as the Vice